

Backend Architecture Planning And Authentication

Module workflow notes

Architecture Overview:

The Raritone system architecture is designed to deliver secure, scalable, and high-performance virtual try-on experiences for mobile users. The Flutter/React Native frontend captures user images and sends them securely to the Node.js + Express.js backend, which handles authentication, request validation, file uploads using Multer, and Cloudinary integration for media management.

Cloudinary optimizes and delivers images through its CDN for faster performance and reduced bandwidth usage, while MongoDB stores only metadata such as secure image URLs, user references, and try-on details instead of raw image files. This architecture improves scalability, security, and overall application performance while supporting future AI-based virtual try-on enhancements.

1. User Registration Workflow

Workflow:

1. User enters registration details
2. Backend validates user data
3. Password is securely hashed
4. User information stored in MongoDB
5. JWT token generated
6. Registration successful response sent

Features:

Secure password handling
User account creation
JWT token generation
MongoDB user storage

2. User Login Workflow

Workflow:

1. User enters email and password
2. Backend validates credentials
3. Password compared securely
4. JWT token generated
5. Token returned to frontend
6. User authenticated successfully

Features

Secure authentication
JWT-based login
Protected backend access
User session handling

3. Protected Route Workflow

Workflow

1. Frontend sends JWT token in request headers
2. Auth middleware receives token
3. Backend verifies JWT token
4. User authorization validated
5. Access granted to protected API

Features

JWT verification
Route protection
Secure API access
Middleware authorization

4. Authentication Middleware Workflow

Workflow

1. Request enters protected route
2. Auth middleware checks authorization header
3. JWT token extracted
4. Token verified using secret key
5. User information attached to request
6. Request proceeds to controller

Features

Secure middleware validation

Unauthorized access prevention

Backend security layer

5. Authentication Technologies Used

Component Technology:

Authentication JWT

Backend Framework Node.js + Express

Database MongoDB

Password Security bcrypt

Middleware Express Middleware

6. Authentication Features Implemented

User Registration

User Login

JWT Authentication

Protected Routes

Auth Middleware

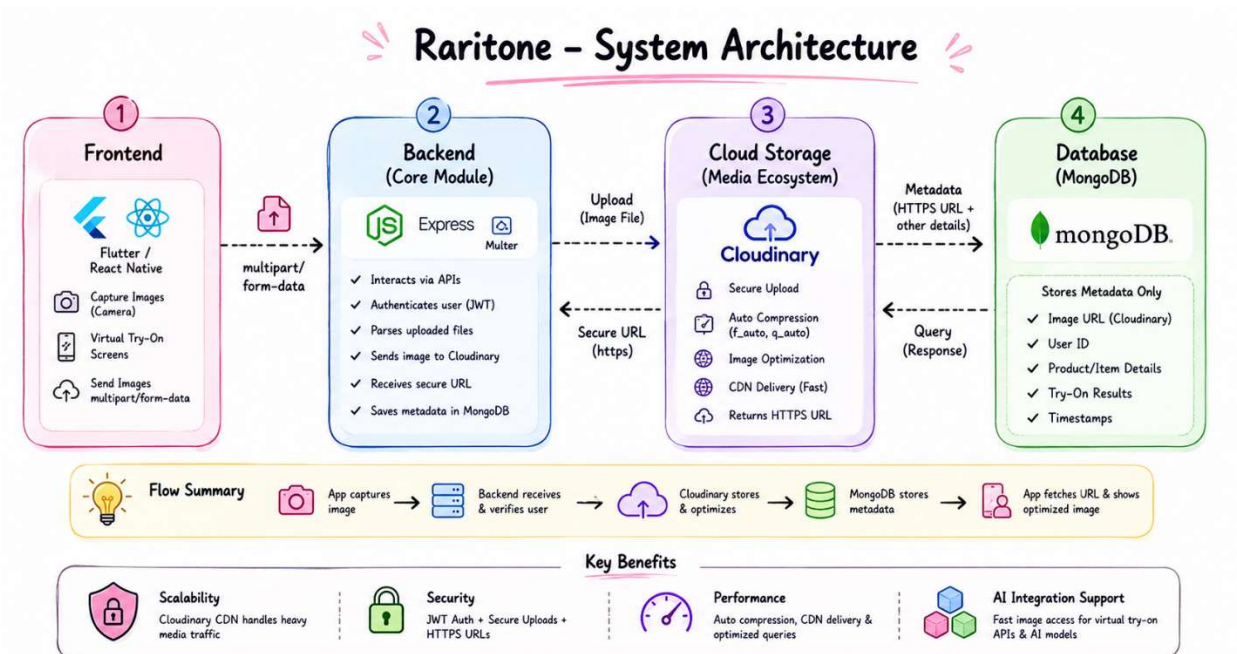
Password Hashing

Secure Token Verification

User Authorization

This architecture emphasizes:

- **Scalability** through CDN delivery and cloud-based storage
- **Security** using JWT authentication and secure HTTPS asset URLs
- **Performance** with optimized media delivery and lightweight database design
- **AI Integration Support** for future virtual try-on and recommendation engine enhancements



The overall system ensures efficient media handling, faster response times, and a clean separation of responsibilities between services, making it suitable for production-scale mobile applications.